

Problem Statement or Requirement:

A requirement from the Hospital, Management asked us to create a predictive model which will predict the **chronic kidney disease** (CKD) based on the several parameters. The Client has provided the dataset of the same.

1.) Identify your problem statement

After reading problem statement it lies “**Machine Learning-Supervised - Classification**” problem because we have to find categorical data (yes or no) and the dataset has been given and we know clearly what we have to predict so its supervised learning behaviour

2.) Tell basic info about the dataset (Total number of rows, columns)

The given dataset is in csv file and it has **28 columns and 399 rows**, so we have to split it into input and output. Except output we have 27 nos of columns as input.

3.) Mention the pre-processing method if you're doing any (like converting string to number – nominal data)

Yes, we use preprocessing method like **one_hot_encoding** for convert our categorical data into numerical one

4.) What are the machine learning algorithms we are going to use?

Name of machine learning algorithm we are using to predict model

SI.NO	NAME_OF_ALGORITHMS	WHAT?	WHY?	HOW?	WHEN?
1	Random Forest	Ensemble of decision trees	Reduces overfitting and improves accuracy	Builds multiple trees on random subsets and averages their predictions	When you want high accuracy and robustness, especially with tabular data
2	Decision Tree	Tree-like model of decisions	Easy to interpret and visualize	Splits data based on feature thresholds to form branches and leaves	When interpretability is key or for quick prototyping
3	SVM (Support Vector Machine)	Finds the best boundary	Effective in high-dimensional spaces	Maximizes margin between	When data is well-separated or for

SI.NO	NAME_OF_ALGORITHMS	WHAT?	WHY?	HOW?	WHEN?
		between classes		classes using support vectors	text/image classification
4	Logistic Regression	Predicts probability of class membership	Simple, fast, and interpretable for binary classification	Uses sigmoid function to model probability between 0 and 1	When you need a baseline model or probability estimates
5	KNN (K-Nearest Neighbors)	Classifies based on closest data points	No training phase; adapts to data naturally	Finds 'k' nearest neighbors and assigns majority class	When data is small and you want a simple, intuitive model
6	Multinomial Naive Bayes	Naive Bayes for discrete count features	Great for text classification (e.g., spam detection)	Calculates probabilities using word frequencies and Bayes' theorem	When working with word counts or document classification
7	Bernoulli NB	Naive Bayes for binary/Boolean features	Useful for binary feature sets like presence/absence of words	Models each feature as a binary variable and applies Bayes' theorem	When features are binary (e.g., word presence in emails)
8	Categorical NB	Naive Bayes for categorical features	Handles non-numeric categorical data directly	Uses category likelihoods and Bayes' theorem	When features are purely categorical (e.g., color, brand)
9	Complement NB	Variant of MultinomialNB for imbalanced data	Improves performance on imbalanced datasets	Adjusts feature weights using complement class statistics	When classes are imbalanced and you're doing text classification

5.) Tabulation of each algorithm and its accuracy

SI_NO	NAME OF ALGORITHMS	LABE PARAMETERS (USED FOR MODEL CREATION)	ACCURACY
1	Random Forest	{'max_depth': None, 'max_features': 'log2', 'min_samples_split': 5, 'n_estimators': 50}	98%
2	Decision Tree	{'class_weight': None, 'criterion': 'gini', 'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 5}	92%
3	SVM	{'C': 1, 'class_weight': 'balanced', 'gamma': 'scale', 'kernel': 'linear'}	98%
4	Logistic Regression	{'class_weight': None, 'fit_intercept': True, 'max_iter': 100, 'multi_class': 'multinomial', 'penalty': 'l2', 'solver': 'newton-cg'}	99%
5	KNN	{'metric': 'manhattan', 'n_neighbors': 3, 'p': 1, 'weights': 'uniform'}	97%
6	Multinomial Navi baye's	{'alpha': 0.1, 'fit_prior': True}	99%
7	GaussianNB	{'var_smoothing': 1e-09}	95
8	BernoulliNB	{'alpha': 0.5, 'binarize': 0.0, 'fit_prior': True}	98%
9	CategoricalNB	{'alpha': 0.1, 'fit_prior': True}	99%
10	ComplementNB	{'alpha': 0.1, 'norm': False}	99%

Random Forest

	precision	recall	f1-score	support
CKD_No	0.98	0.98	0.98	51
CKD_Yes	0.99	0.99	0.99	82
accuracy			0.98	133
macro avg	0.98	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

Decision Tree_Classification

	precision	recall	f1-score	support
DT_CKD_NO	0.85	0.98	0.91	51
DT_CKD_YES	0.99	0.89	0.94	82
accuracy			0.92	133
macro avg	0.92	0.94	0.92	133
weighted avg	0.93	0.92	0.93	133

SVM_Classification

	precision	recall	f1-score	support
SVM_CKD_No	0.94	1.00	0.97	51
SVM_CKD_Yes	1.00	0.96	0.98	82
accuracy			0.98	133
macro avg	0.97	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

LOGISTIC_REGRESSION

	precision	recall	f1-score	support
CKD_LR_NO	1.00	0.98	0.99	51
CKD_LR_YES	0.99	1.00	0.99	82
accuracy			0.99	133
macro avg	0.99	0.99	0.99	133
weighted avg	0.99	0.99	0.99	133

KNN

	precision	recall	f1-score	support
CKD_KNN_NO	0.93	1.00	0.96	51
CKD_KNN_YES	1.00	0.95	0.97	82
accuracy			0.97	133
macro avg	0.96	0.98	0.97	133
weighted avg	0.97	0.97	0.97	133

6.) **Best model for our CKD_Dataset is “Logistic_Regression” because ROC AUC Score is 0.999.**

And the reasons are...

- 1. **Highest ROC AUC Score**
 - [The model achieved a ROC AUC score of 0.99976]
- 2. **Robust Confusion Matrix**
 - (Confusion Matrix: [[50, 1], [0, 82]]) Only **1 misclassification** out of 133 samples, showcasing high precision and recall across both classes
- 3. **Efficient and Scalable**
 - With optimized parameters (solver='newton-cg', penalty='l2', multi_class='multinomial'), the model is both computationally efficient and scalable to larger datasets.